

SAKSHAM AI

Autonomous Vulnerability Auditing & Risk Mitigation Framework

Complete Project Analysis — Requirements, Comparison, Costing, Data & Strategy

Problem Statement: SIH26163 — Security Assessment of the World Monitor application
Organization: National Technical Research Organisation (NTRO)
Category: Software | Theme: Smart Automation (Miscellaneous)
Team Name: PARTH

Live: Backend <https://ai-saksham.onrender.com> | Frontend <https://ai-saksham-mocha.vercel.app>

1. Critical Correction: Problem Statement ID

WARNING: The PPT title slide says SIH-26136, but the official SIH 2026 list records the World Monitor statement as SIH26163 (NTRO). The app code already says NTRO PS-26163. Verify on sih.gov.in and fix the PPT first page — judges catch this instantly.

Official record (verified): SIH26163 · Security Assessment of the World Monitor application · Software / Miscellaneous · NTRO.

2. What Is World Monitor & Why a Framework?

What is World Monitor?

World Monitor is NTRO's web/mobile platform for real-time monitoring, analytics and reporting. It handles user authentication, data visualization, API communication and role-based access control — systems carrying sensitive operational data, where a breach compromises confidentiality, integrity and availability at national scale.

Why did SIH ask for a FRAMEWORK, not a one-time audit?

- Monitoring apps change daily — new endpoints, roles, API versions. A manual audit report is outdated the day it is written.
- NTRO needs a machine that re-audits itself: autonomous, repeatable, evidence-backed. That is the literal meaning of “Auditing and Risk Mitigation Framework”.
- Students cannot be given production access — hence the constraints: authorized systems only, no production impact, PoC-limited validation, legal compliance. A framework with built-in authorization guards satisfies this; ad-hoc hacking does not.
- Saksham AI is exactly that machine — this sentence is the strongest line for the judges.

3. PS Requirements vs Implementation (Verified Against Codebase)

3.1 Scope coverage (all 7 areas)

Authentication & session mgmt	Auth module: JWT, Google OAuth, session verify, demo RBAC logins	Done
Authorization & access control	3-tier RBAC (Admin/Analyst/Viewer) + IDOR ownership tests	Done
Input validation & data handling	Crawler payload analysis, method/endpoint checks	Done
API security	Endpoint discovery, CORS, rate-limit, method checks	Done
Client-side security controls	Security headers audit: CSP, HSTS, X-Frame, MIME	Done
Secure communication	HTTPS/HSTS enforcement checks	Done
Data storage & privacy	Evidence handling, report access control	Done

3.2 The 4 mandated tasks

1. Identify vulnerabilities	8-stage pipeline: reconnaissance > endpointDiscovery > technologyAnalysis > securityChecks > verification > aiAnalysis > riskScoring > reportGeneration
2. Assess potential impact	CVSS v3.1 scoring per finding + AI business-impact assessment + 0-100 security score
3. PoC in controlled environment	Raw HTTP trace captured per finding + one-click PoC cURL generator + safe non-damaging probes only
4. Recommend remediation	Exact remediation steps + AI-generated code fix guidance + hardened config suggestions in every finding and PDF

3.3 Success criteria & deliverables

- At least one valid vulnerability documented — scanner ships with checks for CSP, HSTS, X-Frame, CORS wildcard, rate-limit headers, IDOR, SQL-error disclosure and more.
- Evidence supports it — every finding carries raw HTTP proof (httpTrace), endpoint, method, evidence text.
- Risk clearly explained — severity + CVSS + AI impact summary in plain language.
- Practical mitigation — remediation array + priority reason + copy-ready fix snippets.
- Per-vulnerability deliverable (title, description, component, CVSS, repro steps, PoC, impact, remediation) maps 1:1 to the Finding data model.
- Constraints honored: authorizationConfirmed gate + SSRF guard + “lawful pentest only” posture — testing stays on authorized targets.

4. Solution Architecture

8-stage autonomous pipeline (server/src/models/Assessment.js, assessmentWorker.js)

- 1 reconnaissance — banner, connectivity, latency | 2 endpointDiscovery — APIs, routes, JS assets, deps
- 3 technologyAnalysis — stack fingerprinting | 4 securityChecks — headers, CORS, misconfigurations
- 5 verification — Potential > Under Review > Verified / False Positive | 6 aiAnalysis — Gemini 2.5 Flash (Groq/OpenRouter failover)
- 7 riskScoring — weighted 0–100 security score | 8 reportGeneration — Executive/Technical/Summary PDFs
- Live Socket.io telemetry streams every stage to the dashboard; assessments run ~1 minute in Standard mode.

Dual-layer intelligence

Layer 1 — Automated live HTTP probing engine (scannerEngine.js). Layer 2 — Gemini AI threat analysis engine (aiService.js). Probing finds facts; AI explains impact and writes fixes.

5. Legacy vs Saksham AI — With Worked Examples

Example A: Missing Content-Security-Policy header

Legacy (Nessus/OpenVAS style): one-line alert — “X-Content-Security-Policy missing.” No proof, no fix value; the analyst must research the correct directive set alone.

Saksham AI: live probe captures the raw HTTP response as evidence that the header is truly absent; assigns CVSS 5.4 + severity; gives the exact strict CSP value to deploy; finding stays Potential until an analyst verifies it. Output = verdict, not just alert.

Example B: CORS wildcard (where legacy false-positives)

Legacy: flags every `Access-Control-Allow-Origin: \*`, fifty identical alerts, analysts start ignoring them — then a real one slips through.

Saksham AI: reflects test Origins to prove whether arbitrary origins with credentials are actually honored; only genuinely exploitable cases become Verified (CVSS 6.1), the rest are marked False Positive with reason. Fix: explicit allow-list (exact snippet given).

Example C: IDOR — broken object-level authorization

Legacy DAST: misses logic flaws entirely; needs a manual pentester creating two users and swapping tokens — hours to days of expert time.

Saksham AI: controlled non-owner credential test against object IDs; on success records VERIFIED (confidence ~96%) with PoC cURL evidence; fix: per-object ownership checks + UUIDs + re-test path in API Tester.

Summary table

Time	Manual audit: days-weeks per target	8-stage pipeline, ~1 min, live progress
False positives	200 noisy alerts; analysts drown	Raw HTTP proof + verification workflow
Attack surface	Known URLs only; SPA/AJAX missed	HTML/JS parsing: APIs, routes, tech auto-mapped
Fix quality	“Vuln found” — analyst researches fix	CVSS + exact remediation + AI code patch
Reports	Static scanner dump	Executive / Technical / Summary PDFs with charts
Safety	No guards; wrong-target risk	Authorization gate + SSRF guard; authorized-only
Cost control	Everyone gets full access	3-tier RBAC; Viewer read-only caps scan bills

6. Costing & Verified Industry Data

What security costs today (indicative market ranges)

Manual pentest (India)	Rs 1-5 lakh per assessment	Weeks of effort; not repeatable per release
Nessus Professional	~\$4,000-5,000 / year	Noisy alerts, no proof, no fix
Burp Suite Professional	~\$450 / year	Best-in-class but fully manual — expert required
OpenVAS / OWASP ZAP	Free	High false positives; dev-unfriendly reports
Saksham AI	~Rs 0 (open stack, Gemini free tier, self-hosted)	Autonomous + proof + fix + report

Breach economics — IBM Cost of a Data Breach Report 2025 (verified)

- Global average breach cost: USD 4.44 million (~Rs 37 crore); costs in India are rising.
- Breaches hidden >200 days: USD 5.01M vs caught <200 days: USD 3.87M — early detection saves ~Rs 10 crore. Continuous autonomous scanning attacks exactly this gap.
- Teams using AI + automation: USD 1.9M lower cost, 80 days faster containment — independent proof for the “AI-powered” claim.

- 97% of AI-related breaches lacked proper access controls — the 3-tier RBAC + Viewer cost-guard is a live answer to this finding.

7. Why This App Will Work (Judge-Ready Arguments)

- 1. Exact PS fit: every mandated deliverable field (title, description, component, CVSS, repro, PoC, impact, remediation) exists in the Finding model — not coincidence, design.
- 2. Safety by design: authorization gate + SSRF guard + PoC-limited probes satisfy NTRO's hardest constraint; most student teams cannot show this.
- 3. Evidence builds instant trust: raw HTTP proof + one-click PoC cURL lets a judge verify a finding in 30 seconds.
- 4. Economics: ~Rs 0 per re-run vs lakhs per manual audit; Viewer role caps AI/network scan bills.
- 5. It is a framework, not an audit: runs on ANY web/API target, repeatable every release — precisely what “framework” in the PS title demands.
- Winning line: “Legacy tools give alerts; Saksham AI gives verdicts — with proof, score and fix.”

8. Pre-Submission Checklist

- Fix PPT slide 1: SIH-26136 > SIH26163 (verify on sih.gov.in); keep app footer as-is (already correct).
- Soften absolute claims: “under 60s” > “about a minute (Standard mode)”; “CG ero-latency” > “instant local feedback”; “eliminating false positives” > “separating verified threats from noise”.
- Run one live scan on an authorized demo target; put real findings + PoC screenshots in the PPT.
- Demo flow (5 min): landing > Start Assessment (quick cURL mode) > live pipeline > Verified finding with proof > PoC cURL > PDF report > Settings theme toggle.
- Keep “framework, not audit” + breach-economics (\$4.44M, 200-day, \$1.9M AI saving) lines ready for Q&A.